

Inteligência Computacional

Aula 13: DBSCAN

Prof. Eduardo Corrêa Gonçalves

07/10/2026

Sumário

Introdução

Famílias de Algoritmos de Clustering

Algoritmo DBSCAN

Definições Preliminares

Como funciona o DBSCAN?

Exemplo passo a passo

Informações Adicionais

[Link para o notebook com o exemplo da aula](#)

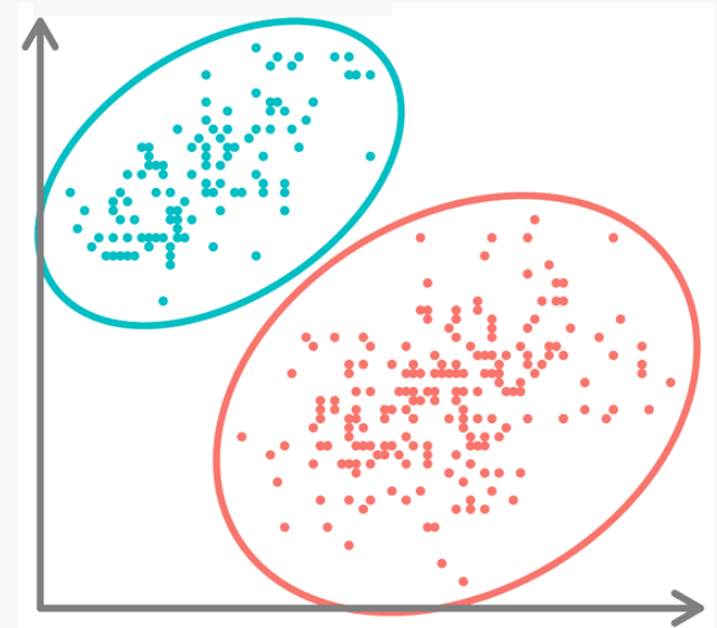
Comparando Algoritmos de Clustering

Introdução (1/5)

- **Clustering (Determinação de Agrupamentos)**
 - Processo não supervisionado em que algoritmo **divide**, de forma automática, um conjunto de objetos em **clusters** (grupos) de acordo **similaridade** entre eles.
 - Na literatura, o *clustering* também é chamado de:
 - Segmentação.
 - Determinação de Agrupamentos.
 - Análise de Agrupamentos.
 - Classificação Não Supervisionada.
 - Aprendizado Não Supervisionado.
 - Taxonomia Automática.
 - Clusterização (*palavra aportuguesada*).
 - Há diferentes **famílias de algoritmos** de clustering. Nos slides a seguir, são apresentadas as principais (*mas há outras como “grafo” e “grid-based”*):

Introdução (2/5)

- Família 1 – Algoritmos Baseados em **Particionamento**
 - Dividem os objetos em **k clusters** de formato esférico.
 - Baseados em **distância**: um objeto irá pertencer ao cluster cujo centro estiver mais próximo.
 - **Heurísticos**: utilizam um mecanismo iterativo de realocação, que tenta melhorar o particionamento movimentando objetos de um cluster para o outro, até chegar a um bom particionamento.
- **Algoritmos**:
 - **k-Means**
 - **k-Medoids (PAM e CLARA)**
 - **k-Modes**
 - **k-Prototypes**
 - **Mini Batch k-Means**



Introdução (3/5)

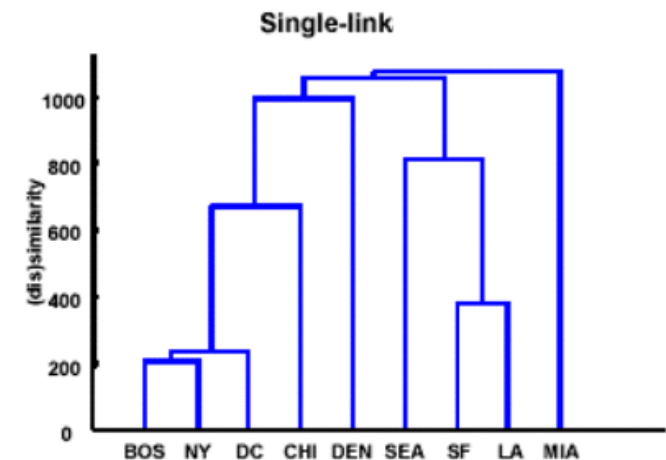
- Família 2 – Algoritmos **Hierárquicos**

- Trabalham agrupando dados em uma hierarquia ou **árvore de clusters**.
- Podem usar a abordagem **aglomerativa** ou **divisiva**, sendo a primeira a mais usada.
 - Abordagem **aglomerativa** (*bottom-up*): começa deixando cada objeto formar seu próprio cluster e iterativamente os combina em clusters maiores e maiores.

- **Algoritmos:**

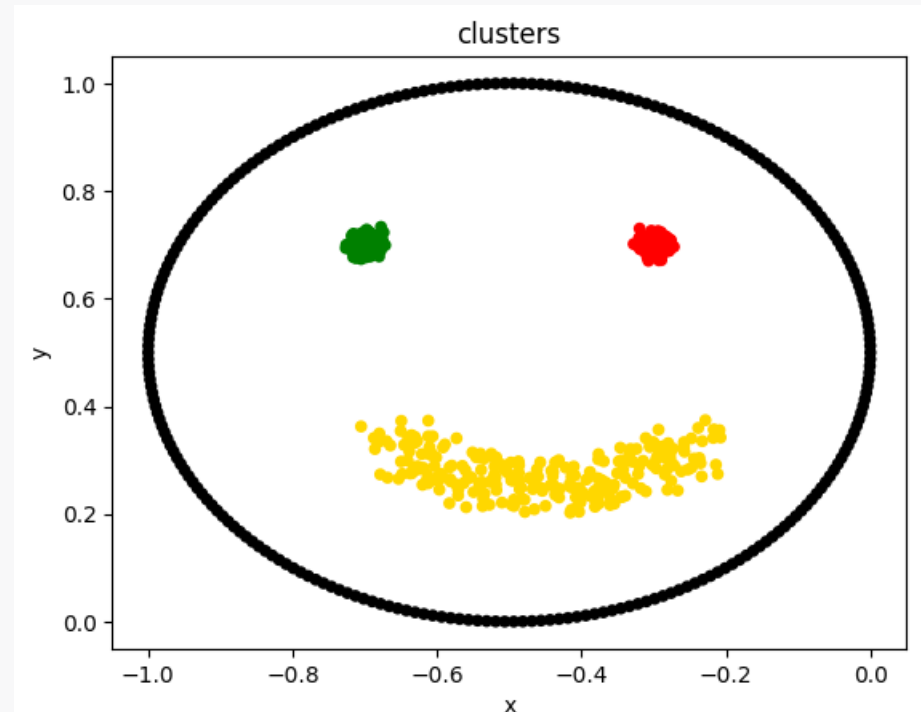
- **AGNES**
- **DIANA**
- **BIRCH**
- **Chameleon**
- **Ward**

	BOS	NY	DC	MIA	CHI	SEA	SF	LA	DEN
BOS	0	206	429	1504	963	2976	3095	2979	1949
NY	206	0	233	1308	802	2815	2934	2786	1771
DC	429	233	0	1075	671	2684	2799	2631	1616
MIA	1504	1308	1075	0	1329	3273	3053	2687	2037
CHI	963	802	671	1329	0	2013	2142	2054	996
SEA	2976	2815	2684	3273	2013	0	808	1131	1307
SF	3095	2934	2799	3053	2142	808	0	379	1235
LA	2979	2786	2631	2687	2054	1131	379	0	1059
DEN	1949	1771	1616	2037	996	1307	1235	1059	0



Introdução (4/5)

- Família 3 – Algoritmos Baseados em **Densidade**
 - Adequados para problemas em que clusters possuem **formato arbitrário** (não esférico).
 - Os clusters representam **regiões densamente populadas** por objetos, separadas por regiões de baixa densidade.
 - **Algoritmos:**
 - **DBSCAN**
 - **HDBSCAN**
 - **OPTICS**



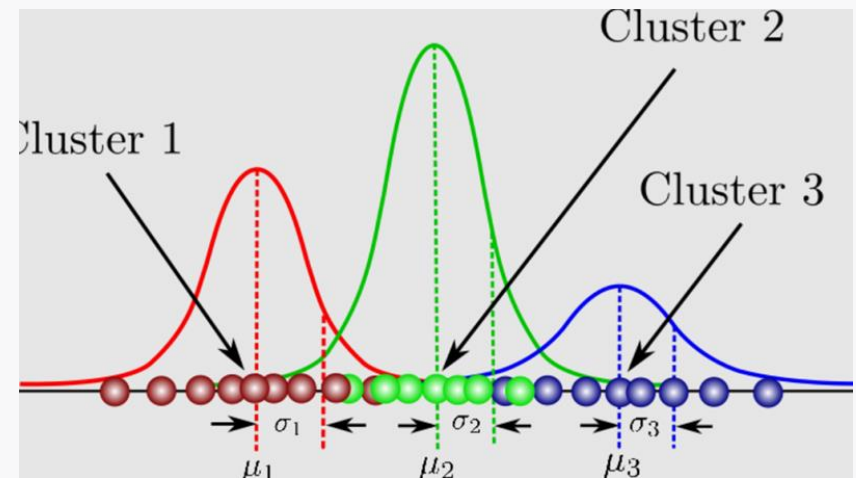
Introdução (5/5)

- **Família 4 – Algoritmos Probabilísticos**

- Utilizam uma **medida probabilística** (em vez de medida de distância ou densidade) para determinar que objetos pertencerão a quais clusters.
- **Exemplo:** pode considerar uma curva em forma de sino para cada dimensão, com uma média e desvio padrão.
 - Um ponto dentro da curva será associado a um cluster com uma determinada probabilidade.
 - Um objeto poderá pertencer a múltiplos clusters, com uma dada probabilidade (*soft clustering*).

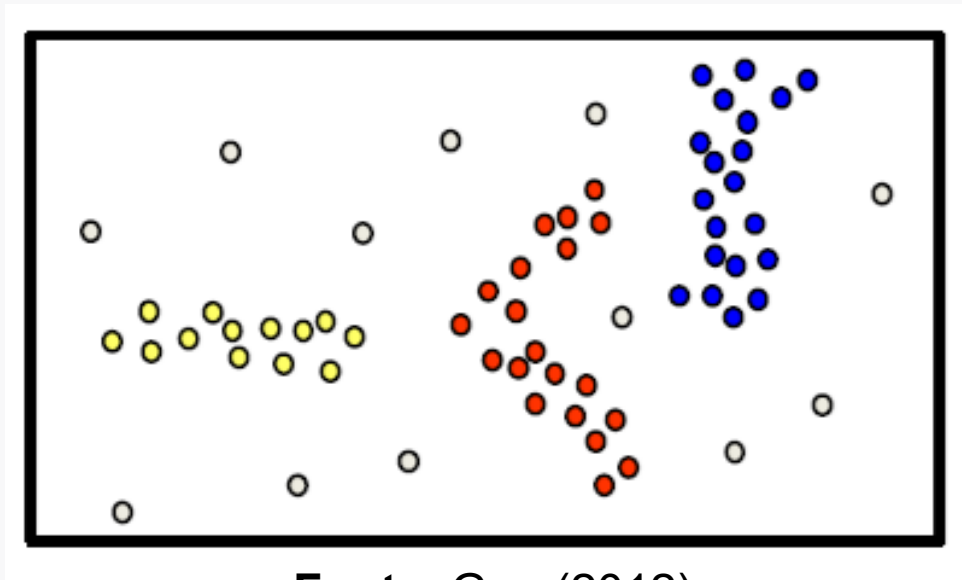
- **Algoritmos:**

- Gaussian Mixture Model / Expectation-Maximization
- Probabilistic Fuzzy C-Means



Apresentando o DBSCAN (1/2)

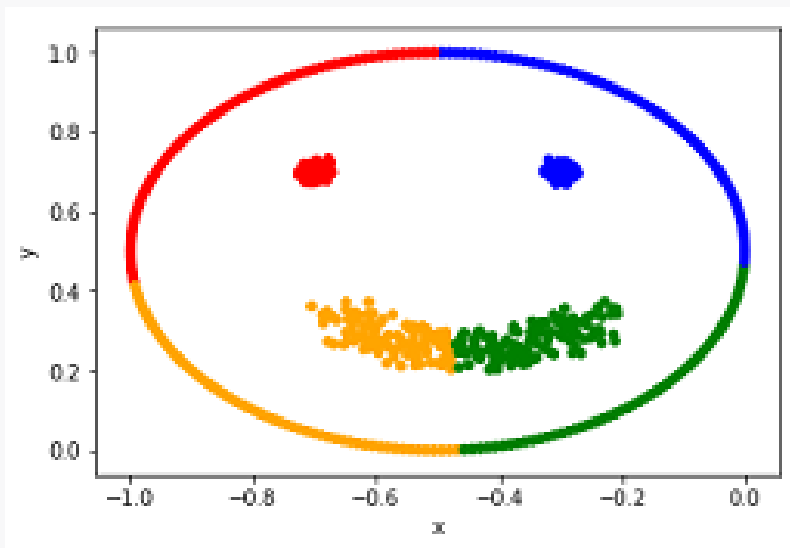
- DBSCAN é o mais famoso algoritmo de *clustering* da família dos métodos de “densidade”
 - O DBSCAN baseia-se na seguinte **hipótese**:
 - Clusters são **regiões densas** no espaço de dados...
 - ... **separadas** por regiões com **menor densidade** de objetos.



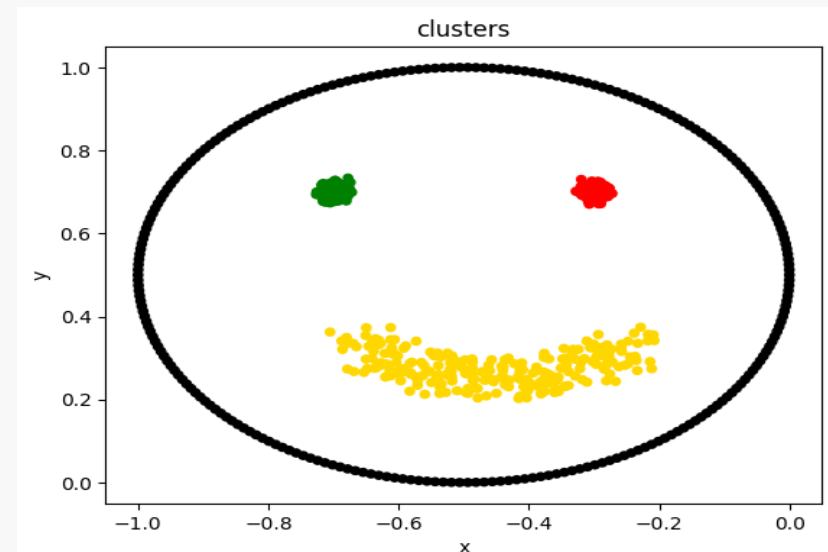
Fonte: Gao (2012)

Introdução (2/2)

- **Pontos Densamente Conectados**
 - Um cluster é definido como o conjunto de máximo de pontos **densamente conectados**.
 - Assim, consegue descobrir clusters com **formato arbitrário**.



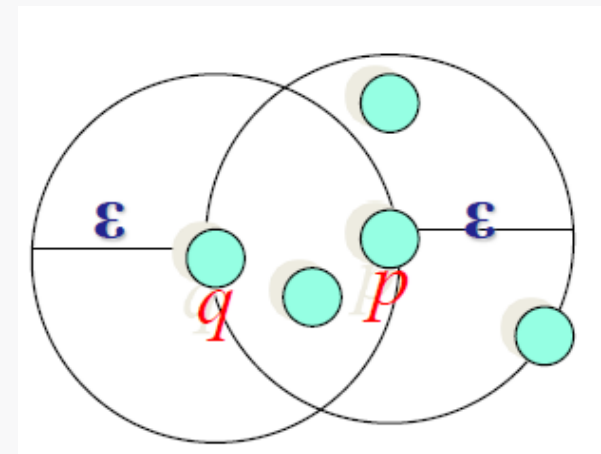
Resultado do k-Means



Resultado do DBSCAN

Definições Preliminares (1/4)

- ε -vizinhança (ε -neighborhood – N_ε)
 - Objetos que estão dentro de um raio ε de distância de um dado objeto p .
 - $N_\varepsilon(p) : \{q \mid d(p, q) \leq \varepsilon\}$
 - A distância $d(p, q)$ pode ser a Euclidiana
- **Alta Densidade**
 - Ocorre quando a ε -vizinhança de um objeto p contém pelo menos $MinPts$ objetos.
 - No exemplo ao lado, para $MinPts = 4$
 - A densidade de p é **alta**.
 - A densidade de q é **baixa**.

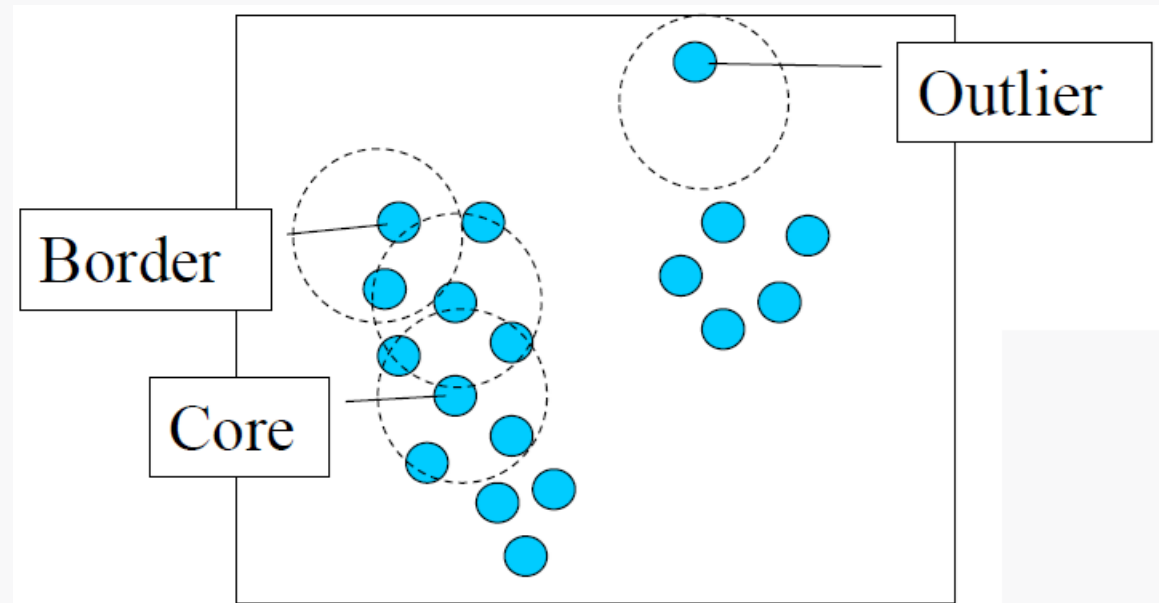


Definições Preliminares (2/4)

- **Tipos de Pontos** – dados ε e **MinPts**, os objetos do BD são classificados em 3 tipos:
 - **core point**: possui número de pontos $\geq \text{MinPts}$ dentro do raio ε . Esses são os pontos no interior do cluster.
 - **edge point** ou **border point**: possui menos de MinPts dentro do raio ε , porém é vizinho de um core point. Ficarão na “borda” do cluster.
 - **noise point** ou **outlier**: qualquer outro ponto.

- No exemplo ao lado:

- $\varepsilon = 1$
- **MinPts** = 4

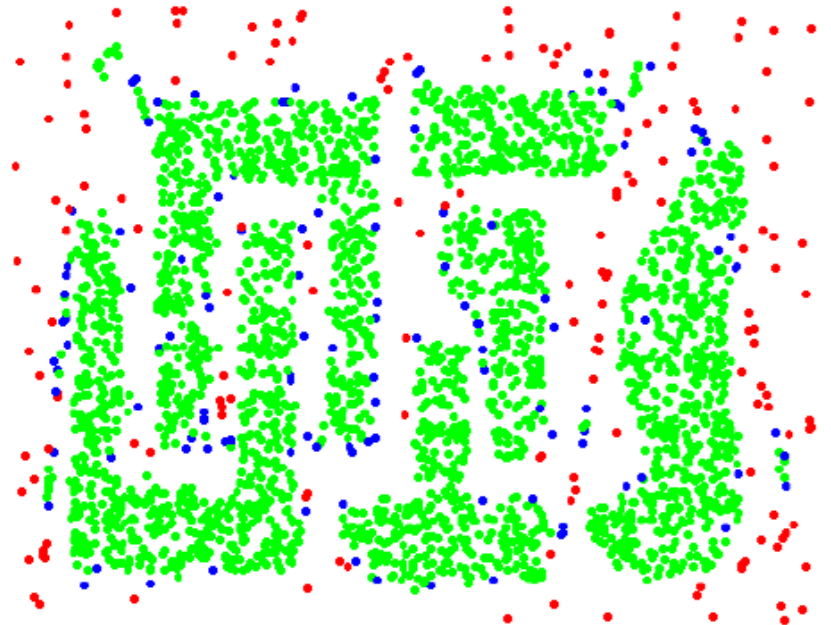


Definições Preliminares (3/4)

- Exemplo



Original Points

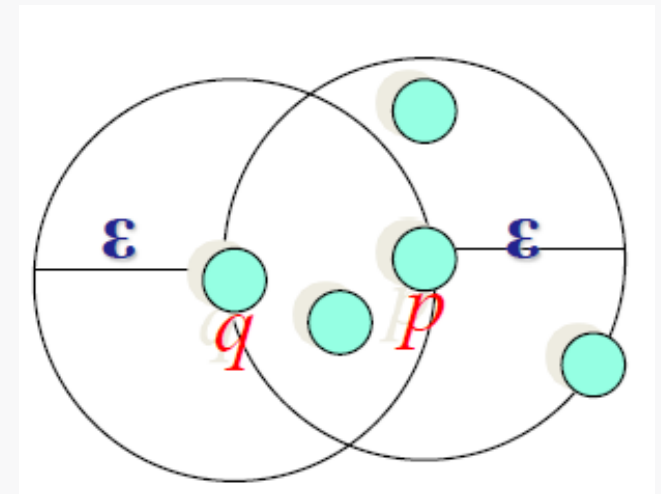


Point types: **core**,
border and **outliers**

Parâmetros: $\varepsilon = 10$ e **MinPts = 4**

Definições Preliminares (4/4)

- **Ponto diretamente alcançável por densidade** (*directly density-reachable point*)
 - Um ponto q é diretamente alcançável por densidade a partir do objeto p se:
 - p for um core point;
 - e q estiver na ε -vizinhança de p .
- No exemplo ao lado, para **MinPts = 4**
 - q é diretamente alcançável por p
 - p não é diretamente alcançável por q
 - O conceito é assimétrico.



Como Funciona o DBSCAN? (1/2)

- Algoritmo DBSCAN

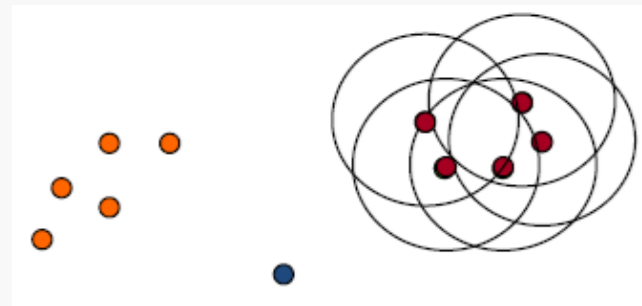
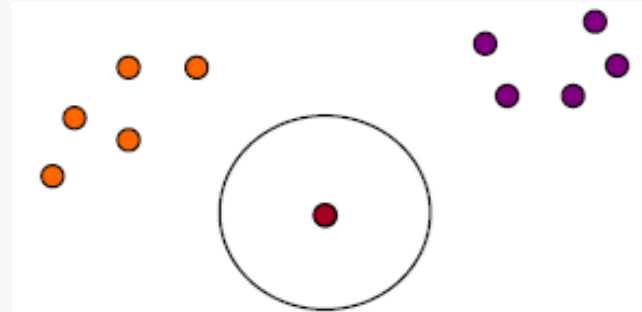
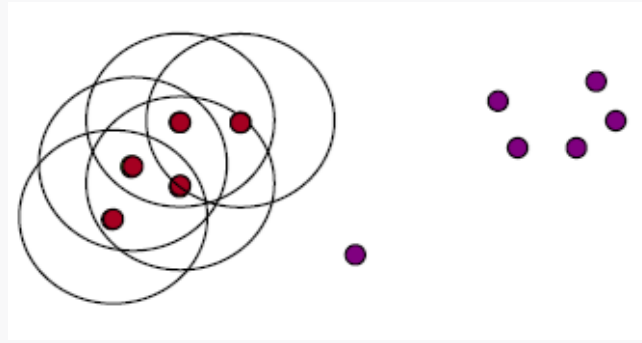
Entrada:

- D – base de dados
- ε – raio
- $MinPts$ – número mínimo de pontos na ε -vizinhança

Algoritmo:

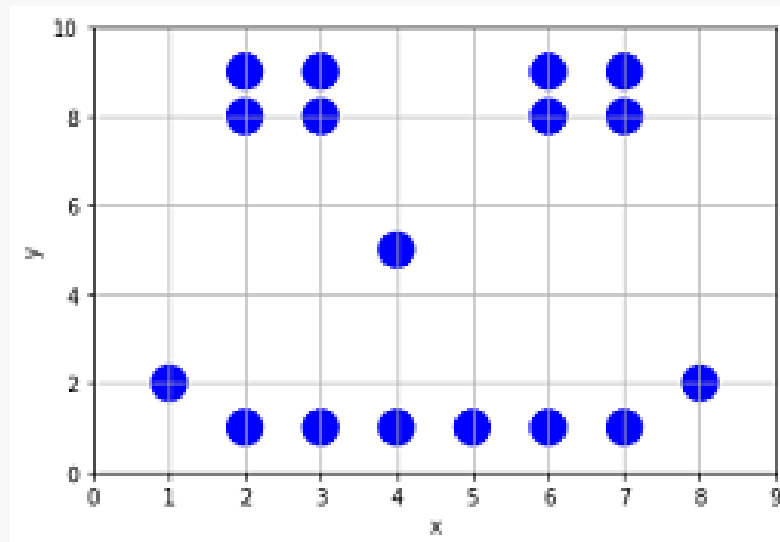
1. para cada ponto $p \in D$ faça:
2. se p ainda não foi tratado **então**:
3. se p é um *core point* **então**:
4. colocar em um novo cluster todos os pontos diretamente alcançáveis por densidade a partir de p ;
5. **senão**:
6. classificar p como um *noise point*;

Como Funciona o DBSCAN? (2/2)



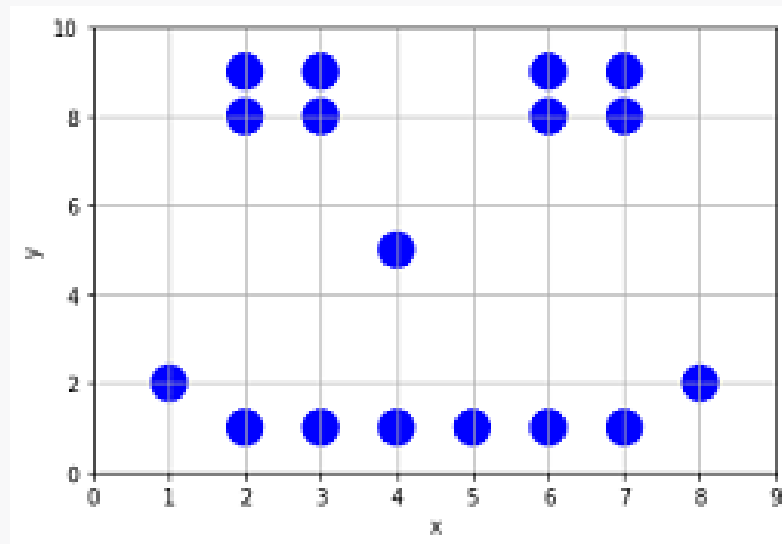
DBSCAN – Exemplo passo a passo (1/6)

- **Exemplo** – como **entrada**, o algoritmo deve receber:
 - Uma base de dados D .
 - O raio ϵ
 - E o número mínimo de pontos $MinPts$
- Demonstraremos o funcionamento do DBSCAN para a base de dados bidimensional abaixo, usando $\epsilon = 1.5$, $MinPts = 2$.



DBSCAN – Exemplo passo a passo (2/6)

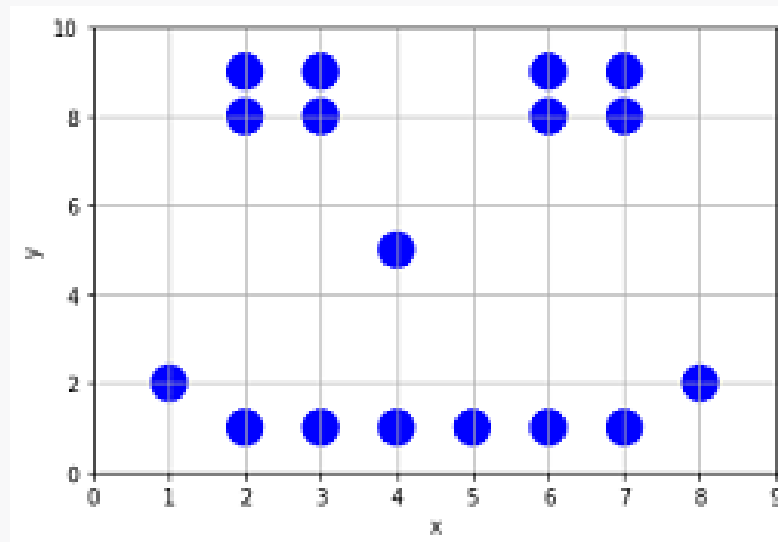
- **Exemplo**
 - 17 objetos.
 - 3 clusters bem definidos
 - 1 ponto isolado na coordenada (4,5)



DBSCAN – Exemplo passo a passo (3/6)

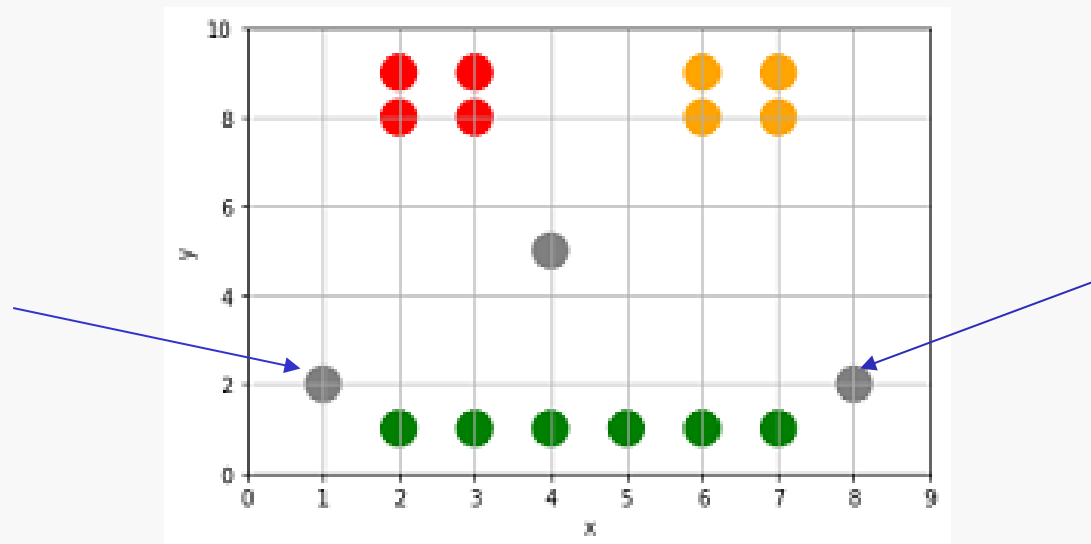
- Exemplo

- O DBSCAN não criará um cluster para o objeto isolado
 - Ele considera mais interessante marcá-lo como **ruído** (*noise point*) .
- Por isso:
 - Apenas *core points* (pontos que contém um número mínimo *MinPts* de vizinhos em seu raio) e *border points* (vizinhos dos *core points*) são levados em consideração.
 - Os demais serão desconsiderados (marcados como ruído).



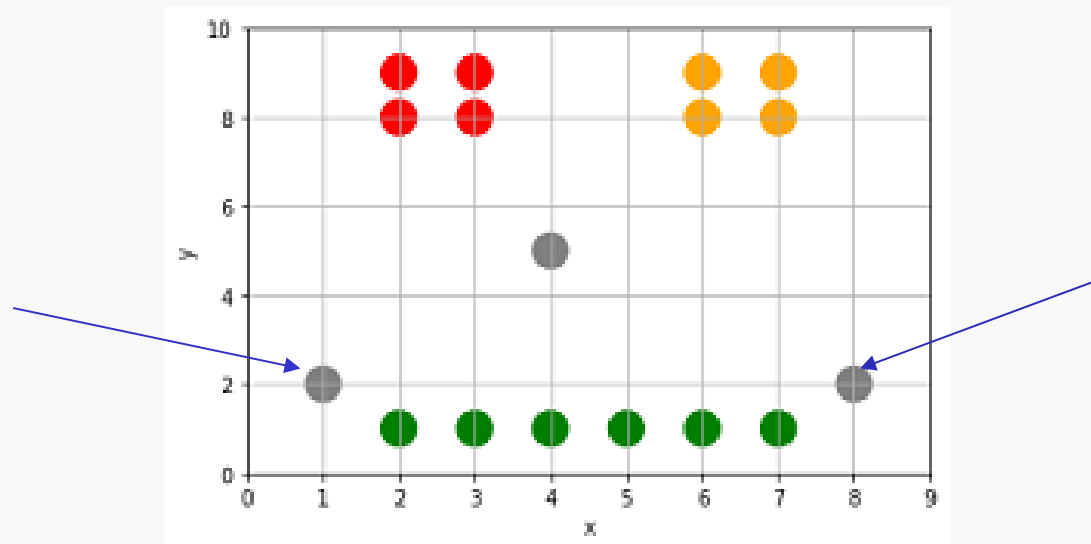
DBSCAN – Exemplo passo a passo (4/6)

- Parâmetros de entrada: $\epsilon = 1.5$; MinPts = 2
 - Significado:
 - Para que um objeto seja *core point*, é preciso que existam 2 vizinhos a ele dentro de um raio de tamanho 1.5.
 - Se o algoritmo fosse “descuidado”, os pontos em cinza abaixo seriam marcados como *noise*, ficando fora do cluster verde



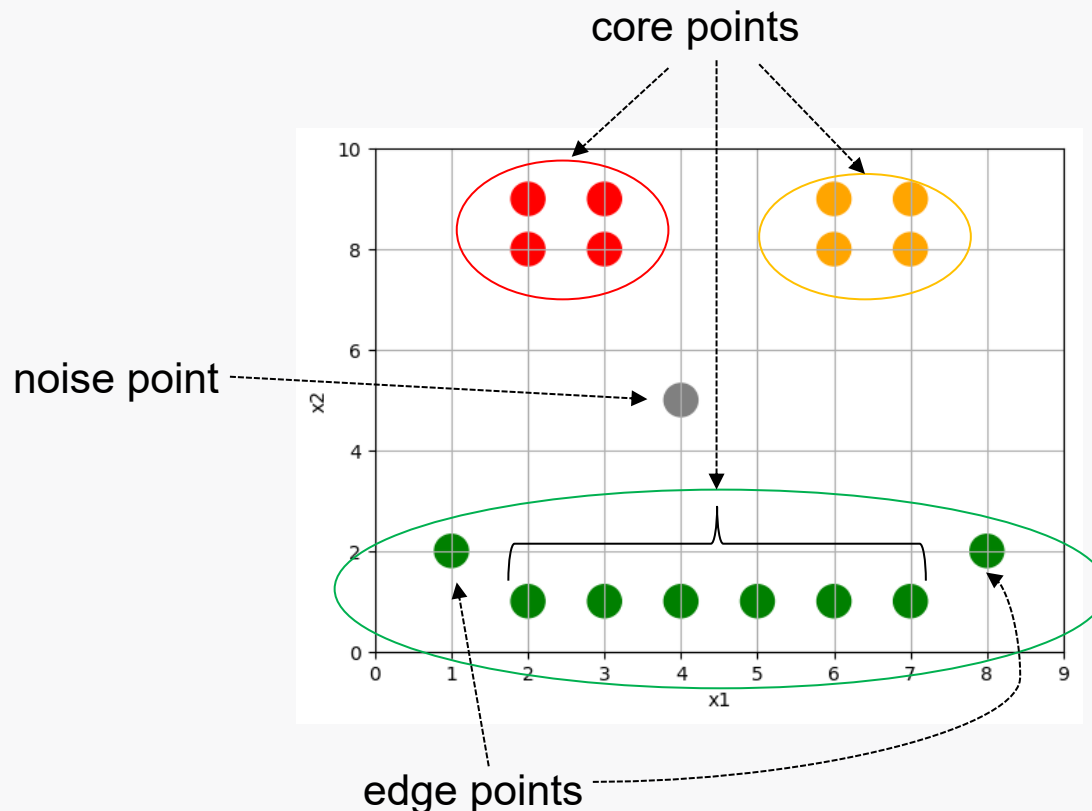
DBSCAN – Exemplo passo a passo (5/6)

- Tratamento dos *edge points*:
 - Os pontos (1,2) e (8, 2) estão na **borda** do cluster verde.
 - O algoritmo foi elaborado de modo a incluir esses pontos da extremidade (*border points*) no cluster.
- Para tal, o DBSCAN define noise point da seguinte forma:
 - “*noise point* é um ponto que não tem um número mínimo de vizinhos *MinPts* e também não está dentro do raio ϵ de um *core point*”.



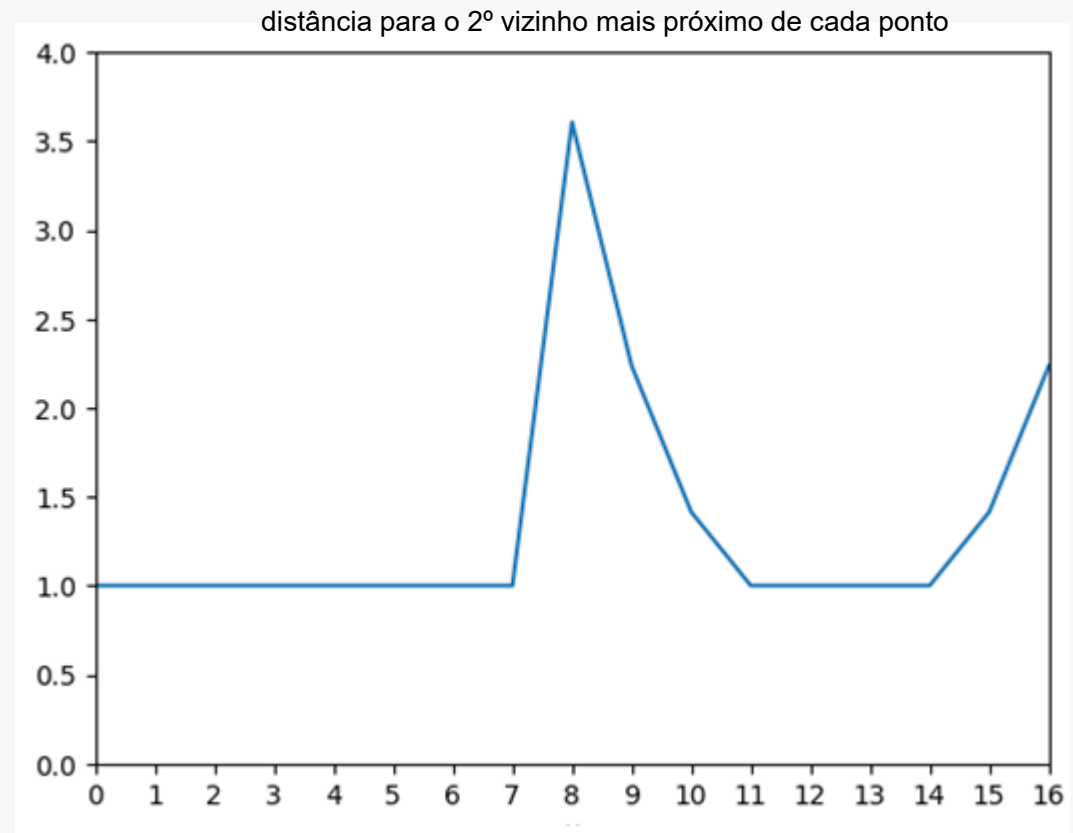
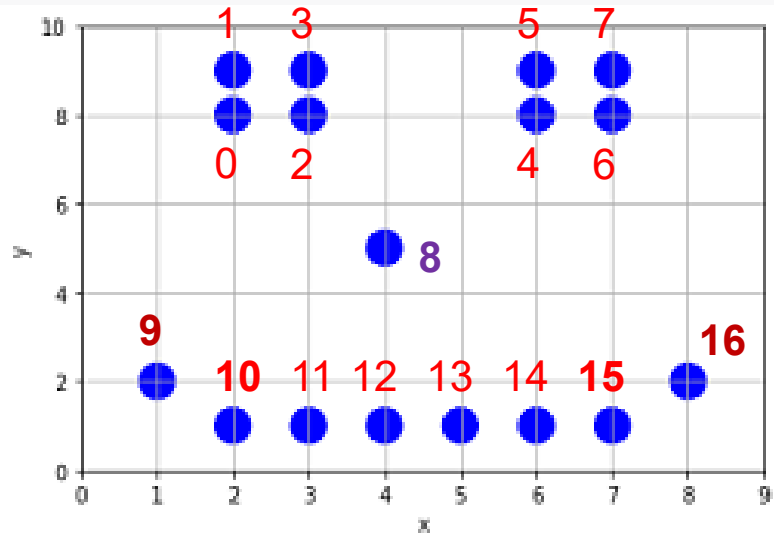
DBSCAN – Exemplo passo a passo (6/6)

- Recordando, há 3 tipos de ponto:
 - Core:** satisfazem o parâmetro *MinPts*
 - Edge:** estão dentro do raio de um *core point*, mas não satisfazem *MinPts*. Vão entrar no cluster.
 - Noise:** não satisfazem o parâmetro *MinPts* e não estão dentro do raio de um *core point*. Não vão entrar em nenhum cluster



DBSCAN – Hiperparâmetros

- Como determinar o valor ideal para ϵ ?
 - Seja $k = \text{MinPts}$. Em um cluster, a distância de um ponto ao seu k -ésimo vizinho será aproximadamente igual para todos os pontos.
 - Já o k -ésimo vizinho de um *noise point* estará a uma distância mais longa.
 - Desta forma, pode-se plotar a distância de cada ponto ao seu k -ésimo vizinho para obter o valor de ϵ . **Problema:** depende de MinPts ...



DBSCAN – Prós e Contras

- **Vantagens do DBSCAN**

- Não é preciso especificar k .
- É capaz de lidar com outliers
 - Inclusive, é muito usado como método para identificar outliers.
- Encontra clusters com formato arbitrário (e não apenas oval)

- **Desvantagens do DBSCAN**

- É sensível aos valores de ϵ e *MinPts*, mas não é fácil determinar o valor ideal de ambos.
- Dificuldade de lidar com situações onde cada cluster tem uma densidade diferente.
- Qualidade do processo não pode ser estimada usando medidas tradicionais como a Silhueta.
 - Deve-se usar outra medida, como por exemplo a DBCV (<https://github.com/christopherjenness/DBCV>)

Comparando Algoritmos (1/3)

- **Aspectos que podem ser usados para comparar algoritmos:**
 - (i) **Escalabilidade e Dimensões Altas**
 - Capacidade ser capaz de lidar com BDs grandes.
 - **Ex.:** clustering de documentos pode lidar com centenas de milhares de palavras.
 - O *k*-Means é facilmente paralelizável (é, portanto, escalável), porém não lida bem com muitas dimensões.
 - (ii) **Habilidade de lidar com diferentes tipos de atributos**
 - A maior parte dos algoritmos trabalha só com atributos numéricos.
 - Um algoritmo que trabalhe de forma natural com atributos nominais pode ser desejável para certos problemas (ex.: *k*-Prototypes).
 - (iii) **Habilidade de descobrir clusters de formato arbitrário**
 - Os métodos de particionamento só conseguem descobrir clusters esféricos
 - Mas em certas aplicações, os clusters tem outros formatos.
 - **Ex.:** identificação de regiões e suas fronteiras a partir de fotos de satélite.

Comparando Algoritmos (2/3)

- **Aspectos que podem ser usados para comparar algoritmos:**
 - (iv) **Exigência de Conhecimento do Domínio para Parametrizar**
 - Alguns algoritmos exigem parâmetros que só podem ser especificados se o usuário tiver conhecimento sobre o problema. **Ex.:** valor de k .
 - *O ideal é que isso não fosse necessário...*
 - (v) **Habilidade de lidar com ruído e outliers**
 - Algoritmos sensíveis a ruído e outliers, como o k -Means, vão produzir clusters pobres (será preciso tratar na fase de pré-processamento)
 - **Ex.:** dados de sensores costumam ter ruídos gerados por falhas e interferências.
 - (vi) **Habilidade de realizar o clustering incremental**
 - Quando novos dados chegam, certos algoritmos exigem que todo o processo seja refeito.
 - Outros, como o k -Means, podem ser facilmente adaptados para modificar os clusters levando em conta apenas o processamento dos objetos novos.

Comparando Algoritmos (3/3)

- **Aspectos que podem ser usados para comparar algoritmos:**
 - (vii) **Outros**
 - Capacidade de produzir hierarquias.
 - Capacidade de realizar o soft clustering (objeto pode pertencer a mais de um clustering)
 - Capacidade de separar outliers em grupo a parte ou fora de todos os grupos.

Link para notebook com os exemplos da aula:

- https://colab.research.google.com/drive/12m005XKd5geZ_DVah_-rmwQsS3KP8qqC?usp=sharing

Referências

- GAO, J. Density-based Clustering. 2012. Disponível em: <<https://cse.buffalo.edu/~jing/cse601/fa12/>>. Acesso em: 12 abr. 2024.
- HAN, J.; KAMBER, M.; PEI, J. Cluster Analysis: Basic Concepts and Methods. *In*: HAN, J.; KAMBER, M.; PEI, J. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kaufmann, 2011. cap 10, p. 471-479.
- SCIKIT-LEARN. k-Means. 2024. Disponível em: <<https://scikit-learn.org/stable/modules/clustering.html#k-means>>. Acesso em: 8 mai. 2024.